



# Post Modification System Testing - CodeSmile

| Autore               | Matricola  |
|----------------------|------------|
| Daniele Pio Scaparra | NF22500101 |
| Grazia Bonagura      | NF22500105 |
| Antonio Nappi        | NF22500200 |

---

## Indice

|                                                                      |          |
|----------------------------------------------------------------------|----------|
| <b>1. Introduzione.....</b>                                          | <b>2</b> |
| <b>2. Post-Modification Testing.....</b>                             | <b>2</b> |
| 2.1 CR1 – Correzione della detection rule in_place_apis_misused..... | 2        |
| 2.2 CR2 – Introduzione della generazione del Call Graph.....         | 4        |
| 2.3 CR3 – Visualizzazione del Call Graph.....                        | 5        |
| <b>3. Regression Testing.....</b>                                    | <b>6</b> |
| 3.1 Obiettivo.....                                                   | 6        |
| 3.2 Strategia.....                                                   | 6        |
| 3.3 Risultati.....                                                   | 6        |
| <b>4. Code Coverage.....</b>                                         | <b>6</b> |
| <b>5. Coerenza con la Impact Analysis.....</b>                       | <b>7</b> |
| <b>6. Conclusioni.....</b>                                           | <b>7</b> |

# 1. Introduzione

Il presente documento descrive le attività di **Post-Modification Testing** e **Regression Testing** effettuate sul sistema *CodeSmile* a seguito dell'implementazione delle **Change Requests CR1, CR2 e CR3**.

Lo scopo del documento è:

- verificare che le modifiche introdotte siano state correttamente implementate;
- garantire che il comportamento preesistente del sistema non sia stato compromesso;
- confermare la coerenza delle attività di test con quanto definito nel **Master Test Plan** e nella **Impact Analysis**.

Le attività di testing sono state condotte mediante test automatici (unit e integration testing) e analisi della copertura del codice.

---

## 2. Post-Modification Testing

Le attività di Post-Modification Testing sono state pianificate in base alle componenti individuate come impattate nel **Documento di Impact Analysis (Doc 6)**.

### 2.1 CR1 – Correzione della detection rule in\_place\_apis\_misused

**Tipo di modifica:** Corrective / Perfective

#### Componenti coinvolte

- *detection\_rules/generic/in\_place\_apis\_misused.py*
  - *test/unit\_testing/detection\_rules/generic/test\_in\_place\_apis\_misused.py*
- 

#### Descrizione della modifica

La CR1 riguarda la correzione e l'estensione della regola di detection *InPlaceAPIsMisused*. In particolare, la modifica ha previsto:

- la revisione della logica di detection per le API **Pandas**;
- l'estensione della regola per includere anche **API NumPy** che restituiscono nuovi oggetti e non operano in-place;
- l'aggiornamento del messaggio descrittivo (*additional\_info*) associato allo smell;
- il mantenimento della compatibilità con i test unitari esistenti, evitando regressioni sul

comportamento precedente.

L'obiettivo della modifica è migliorare la copertura dei pattern di misuse rilevati, riducendo i casi di mancata individuazione (falsi negativi).

---

### **Attività di testing**

Sono stati eseguiti test unitari mirati per verificare:

- la corretta individuazione dello smell quando il risultato di una chiamata non in-place non è assegnato e il parametro inplace non è impostato;
- l'assenza di falsi positivi quando inplace=True oppure quando il risultato è correttamente assegnato a una variabile;
- il comportamento corretto della regola in assenza dell'import della libreria Pandas.

Tutti i test unitari relativi alla regola sono stati rieseguiti con esito positivo.

---

### **Valutazione empirica**

Al fine di valutare l'impatto effettivo della CR1 sul comportamento della regola, è stata condotta una valutazione empirica su un batch etichettato di funzioni Python, contenente:

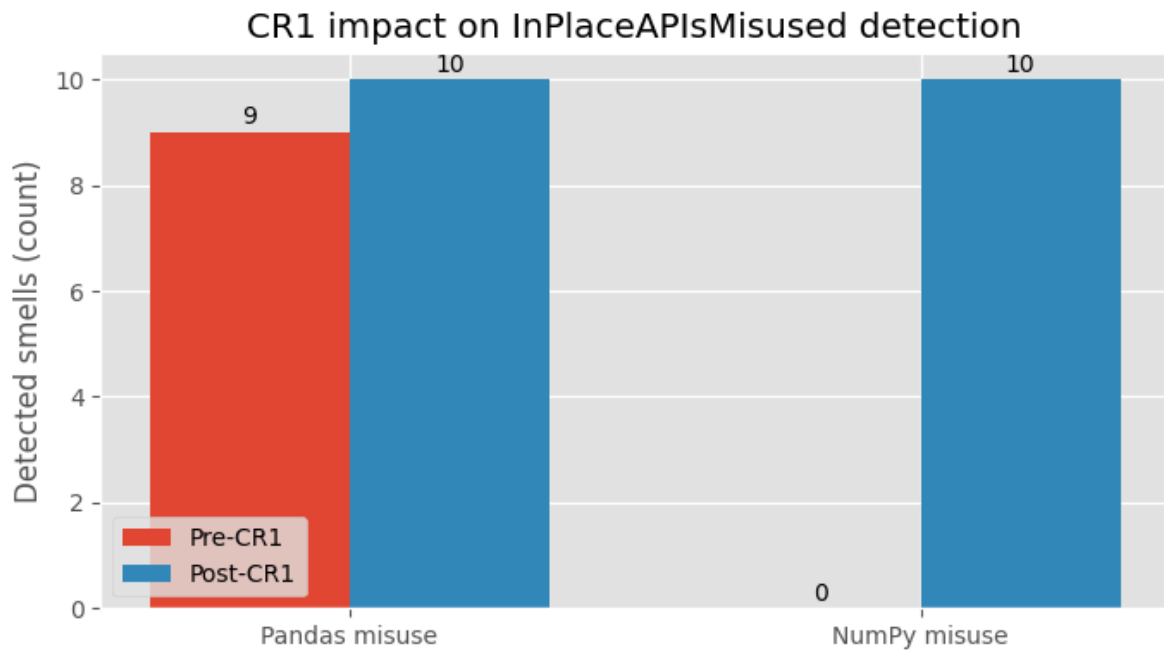
- casi di misuse Pandas;
- casi di misuse NumPy;
- casi corretti (clean).

Lo stesso batch è stato analizzato utilizzando:

- la versione originale della regola (pre-CR1);
- la versione modificata introdotta dalla CR1 (post-CR1).

Il confronto mostra che, nella versione pre-CR1, i casi di misuse NumPy non venivano rilevati, generando falsi negativi. Dopo l'applicazione della CR1, tutti i casi di misuse NumPy inclusi nel batch vengono correttamente individuati.

Il comportamento sui casi Pandas rimane sostanzialmente invariato, indicando che l'estensione non introduce nuovi falsi positivi.



**Fig 1.** Bar Plot dell'impatto della CR1

Come illustrato in **Figura 1**, la modifica comporta una riduzione dei falsi negativi e un conseguente miglioramento della *recall* della regola di detection.

### Esito

Tutti i test unitari associati alla CR1 sono stati superati con successo.

La valutazione empirica conferma che la modifica estende correttamente la copertura della regola *InPlace APIs Misused*, migliorando la capacità di rilevamento dei pattern di misuse senza impatti negativi sul comportamento preesistente.

La CR1 è pertanto considerata **correttamente implementata e validata**.

## 2.2 CR2 – Introduzione della generazione del Call Graph

**Tipo di modifica:** Perfective

**Componenti coinvolte:**

- *cli/cli\_runner.py*
- *components/call\_graph\_generator.py*
- *components/project\_analyzer.py*
- Test unitari e di integrazione associati

**Descrizione della modifica**

La CR2 introduce la possibilità di generare un **call graph** a partire dall'analisi di un progetto, integrando la funzionalità nel flusso CLI esistente.

### **Attività di testing**

Le verifiche effettuate includono:

#### **Unit Testing**

- corretta estrazione di nodi e archi del call graph;
- gestione di grafi con più moduli e chiamate cicliche;
- verifica della struttura dei dati prodotti.

#### **Integration Testing**

- verifica del flusso CLI → ProjectAnalyzer → CallGraphGenerator;
- verifica della compatibilità con le opzioni CLI esistenti;
- validazione dell'output prodotto.

### **Esito**

La funzionalità di generazione del call graph risulta correttamente implementata e integrata nel sistema.

## **2.3 CR3 – Visualizzazione del Call Graph**

**Tipo di modifica:** Perfective

#### **Componenti coinvolte:**

- *components/call\_graph\_analyzer.py*
- *report/report\_generator.py*
- Test unitari e di integrazione associati

#### **Descrizione della modifica**

La CR3 introduce la **visualizzazione del call graph**, arricchendo l'output del sistema con:

- rappresentazione strutturata del grafo;
- metriche associate ai nodi (degree, betweenness, smell count);
- evidenziazione di cicli e hotspot.

La visualizzazione è pensata come supporto all'analisi e alla comprensione della struttura del progetto, senza modificare il comportamento funzionale del sistema.

## Attività di testing

Sono stati eseguiti test per verificare:

- corretto calcolo delle metriche associate ai nodi del grafo;
- corretta individuazione di cicli e hotspot;
- corretta integrazione della visualizzazione nel modulo di reporting;
- comportamento consistente sia in presenza che in assenza della libreria opzionale NetworkX.

## Esito

La funzionalità di visualizzazione del call graph è stata correttamente implementata e validata.

## 3. Regression Testing

### 3.1 Obiettivo

Garantire che le modifiche introdotte dalle CR1, CR2 e CR3 non abbiano introdotto regressioni nel comportamento del sistema.

### 3.2 Strategia

La strategia di regression testing ha previsto:

- riesecuzione completa della suite di test preesistente;
- riesecuzione dei test di integrazione CLI;
- verifica dei risultati rispetto agli output attesi.

### 3.3 Risultati

- Tutti i test sono stati eseguiti con esito positivo.
- Nessuna regressione funzionale è stata rilevata.

---

## 4. Code Coverage

L'attività di testing ha prodotto i seguenti risultati di copertura:

- **Copertura complessiva del progetto: circa 80%**

Le parti non coperte riguardano principalmente:

- rami condizionali legati a dipendenze opzionali;
- codice difensivo e casi limite difficilmente riproducibili in modo deterministico.

Il livello di copertura è considerato adeguato e coerente con gli obiettivi del Master Test Plan.

---

## 5. Coerenza con la Impact Analysis

Le attività di Post-Modification e Regression Testing risultano coerenti con quanto riportato nel **Documento di Impact Analysis**:

- tutte le componenti identificate come impattate sono state testate;
  - la propagazione degli impatti è stata verificata tramite test di integrazione;
  - non sono stati introdotti test su componenti dichiarate fuori scope.
- 

## 6. Conclusioni

Le attività di Post-Modification Testing e Regression Testing sono state completate con successo.

Le modifiche introdotte tramite:

- **CR1**: correzione della detection rule,
- **CR2**: generazione del call graph,
- **CR3**: analisi e visualizzazione del call graph,

sono risultate correttamente implementate, testate e integrate nel sistema.

Il sistema *CodeSmile* può essere considerato **stabile e pronto per il rilascio**.